NUMN33: Project Part 2

Arpita Bhattacharya and Carlotta Boi: Homework Group 6

March 11, 2025

Theoretical Exercises

Task 1. *Let* $\Omega := (-1, 1)^2 \subset R^2$ *and consider the elliptic problem*

$$-\nabla \cdot (\alpha \nabla u) + \gamma u = f \text{ in } \Omega \tag{0.1}$$

$$u = 0 \text{ on } \partial\Omega \tag{0.2}$$

1. State the weak formulation of 0.1.
2. State which functions are reasonable to test with.
3. Choose two suitable (non-constant) functions α *and* γ*. Prove that for your choice of functions and for every* f *in* $H_0^1(\Omega)^*$ *the weak formulation of 0.1 has a unique solution* u *in* $H_0^1(\Omega)$*.*

1. Suppose u is a strong solution and take any test function $v \in C^1(\Omega) \cap C^0(\bar{\Omega})$, then multiplying 0.1 by v, we obtain:

$$-\nabla \cdot (\alpha \nabla u)v + \gamma uv = fv \text{ in } \Omega.$$

By integrating over Ω we have:

$$\int_\Omega (-\nabla \cdot (\alpha \nabla u))v + \int_\Omega \gamma uv dx = \int_\Omega fv dx$$

Since $v = 0$ on $\partial\Omega$ by integrating by parts we obtain:

$$\int_\Omega (-\nabla \cdot (\alpha \nabla u))v dx = \int_\Omega \alpha \nabla u \nabla v dx - \int_{\partial\Omega} \alpha (\nabla u \cdot n)v dS$$

Where the boundary term vanishes due to the homogeneous Dirichlet condition on v, therefore the weak formulation will be: find $u \in H_0^1(\Omega)$ such that:

$$a(u, v) = (f, v), \forall v \in H_0^1(\Omega).$$

$$a(u, v) = \int_\Omega \alpha \nabla u \nabla v dx + \int_\Omega \gamma uv dx = \int_\Omega fv dx, \ \forall v \in H_0^1(\Omega).$$

2. Since u is required to vanish on $\partial\Omega$, i.e. for Dirichlet condition we have $u \in H_0^1(\Omega)$, reasonable functions to test with are those in $H_0^1(\Omega)$.

$$H_0^1(\Omega) = \{v \in H^1(\Omega) : v = 0 \text{ on } \partial\Omega\}$$

In this way the problem:

$$\int_\Omega \alpha \nabla u \nabla v dx + \int_\Omega \gamma u v dx = \int_\Omega f v dx$$

has a unique solution.

3. In order to solve this task, we need Lax - Milgram's lemma and Poincaré's inequality.
Poincaré's inequality. If Ω is bounded, then:

$$||u||_{L^2(\Omega)} \leq const(\Omega)||\nabla u||_{L^2(\Omega)}, \forall u \in H_0^1(\Omega).$$

Note that $u \longrightarrow ||\nabla u||_{L^2(\Omega)}$ and $u \longrightarrow ||u||_{H^1(\Omega)}$ are equivalent norms on $H_0^1(\Omega)$.

Theorem 0.3. *(Lax-Milgram's lemma) Let $(H, (\cdot,\cdot,)_H)$ be a (real) Hilbert space and let $B : H \times H \longrightarrow \mathbb{R}$ be a bilinear functional. If there exists two constants c_1 and $c_2 > 0$ such that*

$$(i)|B(u,v)| \leq c_1||u||_H||v||_H, \ \forall u, v \in H \text{ [Bounded]},$$
$$(ii)|B(u,u)| \geq c_2||u||_H^2, \ \forall u \in H \text{ [Coercive]}.$$

then for every $l \in H^*$ there exists a unique $u \in H$ such that $B(u,v) = l(v), \forall v \in H$.

Denote the bilinear functional as:

$$a(u,v) := \int_\Omega \alpha \nabla u \nabla v + \int_\Omega \gamma u v,$$

and the linear functional as:

$$l(v) := \int_\Omega f v.$$

We choose:

$$\alpha = 1 + x_1^2,$$
$$\gamma = 1 + x_2^2.$$

The functions are positive on Ω, bounded above i.e.

$$1 \leq \alpha \leq 2,$$
$$1 \leq \gamma \leq 2,$$

and they are nonconstant. So we can apply Lax - Milgram's lemma.

Boundedness. Let $u, v \in C_o^\infty(\Omega)$.

$$\begin{aligned}
|a(u,v)| &= |\int_\Omega \alpha \nabla u \nabla v + \int_\Omega uv| dx \\
&\leq \int_\Omega \alpha |\nabla u \cdot \nabla v| + |\gamma||u||v| dx \\
&\text{Using Cauchy -Schwarz on } \mathbb{R}^2\text{: } |w \cdot z| \leq |w| \cdot |z| \\
&\leq max_{x \in \bar{\Omega}}\{\alpha(x), |\gamma(x)|\} \int_\Omega |\nabla u||\nabla v| + |u||v| dx \\
&\text{since } max_{x \in \bar{\Omega}}\{\alpha(x), |\gamma(x)|\} \leq 2, \\
&\text{by Cauchy - Schwarz on } (C_o^\infty(\Omega) \to \int_\Omega uv dx) \\
&\leq \tilde{C}(||\nabla u||_{L^2(\Omega)}||\nabla v||_{L^2(\Omega)} + ||u||_{L^2(\Omega)} + ||v||_{L^2(\Omega)}) \\
&\text{Since } ||u||_{H^1(\Omega)} = \sqrt{||u||^2_{L^2(\Omega)} + ||\nabla u||^2_{L^2(\Omega)}} \\
&\leq C||u||_{H^1(\Omega)}||v||_{H^1(\Omega)}
\end{aligned}$$

Hence a is bounded on $H^1(\Omega) \times H^1(\Omega)$. Now we check the coercivity.

Coercivity. Let $u \in C_o^\infty(\Omega)$:

$$a(u,u) = \int_\Omega \alpha |\nabla u|^2 dx + \int_\Omega \gamma |u|^2 dx.$$

Because $\alpha, \gamma \geq 1 > 0$, we have:

$$\begin{aligned}
&\int_\Omega \alpha |\nabla u|^2 dx \\
\geq min_{x \in \bar{\Omega}}\{\alpha(x)\} \int_\Omega |\nabla u|^2 dx &\geq 1 \cdot ||\nabla u||^2_{L^2(\Omega)} \\
&\geq \frac{1}{2}||\nabla u||^2_{L^2(\Omega)} + \frac{1}{c^2}||u||^2_{L^2(\Omega)}) \\
&\geq c||u||^2_{H^1(\Omega)}.
\end{aligned}$$

And

$$\int_\Omega \gamma |u|^2 dx = \int_\Omega \gamma u^2 dx$$

Therefore we can write

$$a(u,u) = \int_\Omega \alpha |\nabla u|^2 + \gamma |u|^2 dx \geq c||u||^2_{H^1(\Omega)}, \forall u \in C_o^\infty(\Omega).$$

Hence a is coercive.

As $C_o^\infty(\Omega)$ is dense in $H_o^1(\Omega)$ and $\alpha \frac{\partial}{\partial x_i} u$, γu are all elements in $L^2(\Omega)$, when $u \in H_o^1(\Omega)$, we obtain that:

$$a : H_o^1(\Omega) \times H_o^1(\Omega) \to \mathbb{R},$$

is well defined bounded and coercive. Hence the weak form of the problem is $a(u,v) = l(v), \forall v \in H_o^1(\Omega)$, where $l \in H_o^1(\Omega)^*$. Since a is bounded and coercive, we can apply the Lax-Milgram's lemma stated before, thus we obtain that for every $l \in H_o^1(\Omega)^*$ there exists a unique solution $u \in H_o^1(\Omega)$, such that $a(u,v) = l(v), \forall v \in H_o^1(\Omega)$.

Task 2. *Consider the FEM discretization of equation (0.1), and denote the FEM approximation by u_h and the exact solution by u.*
1. Prove that $||u_h||_{H^1(\Omega)} \leq C||u||_{H^1(\Omega)}$.
2. Let $\alpha = \gamma = 1$. *Prove that u_h is then the best possible approximation of u in the FEM space S_{h0} with respect to the norm* $||\cdot||_{H^1(\Omega)}$, *i.e.,*

$$||u - u_h||_{H^1(\Omega)} = inf_{\varphi \in S_{h0}} ||u - \varphi||_{H^1(\Omega)}.$$

To solve this task we use the Galerkin method. In order to obtain a computational approximation $u_h \approx u$, we replace the infinite dimensional space H by a sequence of finite dimensional subspaces:

$$\{H_h\}_{0<h\leq h_{max}},$$

where dim $H_h \longrightarrow \infty$ as $h \longrightarrow 0$ and "$lim_{h\longrightarrow 0} H_h = H$." Next, we replace the weak formulation of our elliptic problem by its Galerkin discretization:

$$\text{Find } u_h \in H_h \text{ such that:}$$
$$B(u_h, \varphi) = l(\varphi), \forall \varphi \in H_h.$$

So for our problem this translates in:

$$\text{Find } u_h \in H_h \text{ such that:}$$
$$a(u_h, v_h) = \int_\Omega f v_h dx, \forall v_h \in H_h.$$

Let u be the exact weak solution of

$$-\nabla \cdot (\alpha \nabla u) + \gamma u = f,$$

and u_h is the FEM approximation satisfying:

$$a(u_h, v_h) = \int_\Omega f v_h dx, \forall v_h \in H_h.$$

Since u satisfies:

$$a(u, v) = (f, v), \ \forall v \in H^1(\Omega)$$

and u_h satisfies:

$$a(u_h, v_h) = (f, v_h), \forall v_h \in H_h$$

we have that for any $v_h \in H_h$:

$$a(u - u_h, v_h) = 0.$$

If we choose $v_h = u_h$, we have:

$$a(u - u_h, u_h) = 0,$$
$$a(u_h, u_h) = a(u, u_h).$$

Using Lax-Milgram's Lemma.
Coercivity of a.: Let

$$a(v, v) = \int_\Omega \alpha |\nabla v|^2 dx + \int_\Omega \gamma v^2 dx,$$

then we have

$$\int_\Omega \alpha |\nabla v|^2 dx \geq min_{x\in\bar{\Omega}}\{\alpha(x)\}(\int_\Omega |\nabla v|^2 dx)$$
$$\geq \alpha_0(||\nabla v||^2_{L^2(\Omega)}) \geq \frac{\alpha_0}{2}(||\nabla v||^2_{L^2(\Omega)} + \frac{1}{c_0^2}||v||^2_{L^2(\Omega)}) \geq c_0||v||^2_{H^1(\Omega)},$$

if $\gamma \geq 0$ and $\alpha \geq \alpha_0 > 0$. Therefore:

$$a(v,v) = \int_\Omega \alpha |\nabla v|^2 dx + \int_\Omega \gamma v^2 dx \geq c_0 ||v||^2_{H^1(\Omega)},$$

$$a(v,v) \geq c_0 ||v||^2_{H^1(\Omega)}.$$

Boundedness of a. We know that:

$$|a(v,w)| \leq c_1 ||v||_{H^1(\Omega)} ||w||_{H^1(\Omega)}.$$

Consequently we obtain:

$$a(u_h, u_h) = a(u, u_h) \leq c_1 ||u||_{H^1(\Omega)} ||u_h||_{H^1(\Omega)}.$$

Using coercivity, $a(u_h, u_h) \geq c_0 ||u_h||^2_{H^1(\Omega)}$, and boundedness, we have that:

$$c_0 ||u_h||^2_{H^1(\Omega)} \leq a(u_h, u_h) \leq c_1 ||u||_{H^1(\Omega)} ||u_h||_{H^1(\Omega)}.$$

Therefore

$$||u_h||_{H^1(\Omega)} \leq \frac{c_1}{c_0} ||u||_{H^1(\Omega)},$$

that we can write as:

$$||u_h||_{H^1(\Omega)} \leq C ||u||_{H^1(\Omega)}.$$

In order to solve 2.2, we use Céa's Lemma:

Lemma 0.4.

$$||u - u_h||_H \leq \frac{c_1}{c_2} inf ||u - \varphi||_H.$$

Let $\alpha = \gamma = 1$. Let a(u,v) be:

$$a(u,v) = \int_\Omega \nabla u \cdot \nabla v dx + \int_\Omega uv dx$$

and let $l(v)$ be:

$$l(v) = \int_\Omega fv dx.$$

Therefore:

$$a(u,v) = l(v), \forall v \in H,$$

and

$$a(u_h, v) = l(v), \forall v \in H_h.$$

Then

$$a(u,v) - a(u_h, v) = a(u - u_h, v) = l(v) - l(v) = 0, \forall v \in H_h.$$

$$\begin{aligned} c_2 ||u - u_h||^2_{H^1(\Omega)} &\leq a(u - u_h, u - u_h + v - v) \\ &= a(u - u_h, u - v) + a(u - u_h, v - u_h) \\ &\leq c_1 ||u - u_h||_{H^1(\Omega)} ||u - v||_{H^1(\Omega)}. \end{aligned}$$

Consequently:

$$||u - u_h||_{H^1(\Omega)} \leq \frac{c_1}{c_2} ||u - v||_{H^1(\Omega)}, \forall v \in H_h$$

and

$$||u - u_h||_{H^1(\Omega)} \leq \frac{c_1}{c_2} inf_{v \in H_h} ||u - v||_{H^1(\Omega)}.$$

For $\alpha = \gamma = 1$, the bilinear form is the $H^1(\Omega)$ inner product, and u_h is the orthogonal projection of u onto S_{ho} with respect to the inner product $a(u, v)$, therefore $c_1 = c_2 = 1$ and so:

$$||u - u_h||_{H^1(\Omega)} = inf_{v \in S_{ho}} ||u - v||_{H^1(\Omega)}.$$

Note that in this case:

$$a(u, v) = (u, v)_{H^1(\Omega)} \text{ and } ||u||_{H^1(\Omega)} = \sqrt{a(u, u)}.$$

Practical Exercises

Task 3. *Write a class LinearLagrangeSpace that provides 3 methods:*

```
class LinearLagrangeSpace:
def __init__(self,view):
self.view = view
# TODO: Create a mapper for vertex indices
self.mapper = see Lecture notes
self.localDofs = 3
self.points = numpy.array( [ [0,0],[1,0],[0,1] ] )
def evaluateLocal(self, x):
# TODO: Return a numpy array with the evaluations
# of the 3 basis functions in local point x
# return numpy.array( [] )
pass
def gradientLocal(self, x):
# TODO: Return a numpy array with the evaluations
# of the gradients of the 3 basis functions in local point x
# return numpy.array( dbary )
pass
```

Our code is:

```
import numpy
from numpy import sin
from dune.fem.function import gridFunction
from dune.grid import cartesianDomain, yaspGrid

domain = cartesianDomain([0, 0], [1, 1], [10, 10])
yaspView = yaspGrid(domain)
from dune.alugrid import aluConformGrid
aluView = aluConformGrid(domain)
aluView.plot()



view = aluView
```

```
class LinearLagrangeSpace:
    def __init__(self,view):
        self.view=view
#TODO: Create a mapper for vertex indices

        self.mapper = view.mapper(lambda gt: 1 if gt.dim == 0 else 0)

        self.localDofs = 3
        self.points = numpy.array([
            [0, 0],
            [1, 0],
            [0, 1]
        ])
    def evaluateLocal(self, x):
    # TODO: Return a numpy array with the evaluations
   # of the 3 basis functions in local point x
   # return numpy.array( [] )
        phi0 = 1-x[0]-x[1]
        phi1 = x[0]
        phi2 = x[1]
        return numpy.array([phi0,phi1,phi2])
        pass

    def gradientLocal(self, x):
# TODO: Return a numpy array with the evaluations
# of the gradients of the 3 basis functions in local point x
# return numpy.array( dbary )

        grad0 = numpy.array([-1, -1])
        grad1 = numpy.array([ 1, 0])
        grad2 = numpy.array([ 0, 1])
        return numpy.array([grad0, grad1, grad2])
        pass
```

We create first a mapper:

```
    self.mapper = view.mapper(lambda gt: 1 if gt.dim == 0 else 0).
```

Where λ tells the mapper to assign DOFs (vertices) to grid entities. The triangle with vertices:

$$v_0 = (0,0),\ v_1(1,0),\ v_2(0,1),$$

is our reference element. We define shape functions on it. For Lagrange functions we use barycentric coordinates:

$$\begin{aligned}\hat{\varphi}_0(\hat{x}) &= \lambda_0(\hat{x}) = 1 - \hat{x_1} - \hat{x_2},\\ \hat{\varphi}_1(\hat{x}) &= \lambda_1(\hat{x}) = \hat{x_1},\\ \hat{\varphi}_2(\hat{x}) &= \lambda_2(\hat{x}) = \hat{x_2}.\end{aligned}$$

Our snippet:

```
    evaluateLocal(self, x):
        phi0 = 1-x[0]-x[1]
        phi1 = x[0]
        phi2 = x[1]
        return numpy.array([phi0,phi1,phi2])
```

implements the evaluation of $\hat{\varphi}_i$ at $\hat{x}$. We will have that the basis functions are:

$$\hat{\varphi}_0(\hat{x}) = \hat{\varphi}_0(x, y) = 1 - x - y,$$
$$\hat{\varphi}_1(x, y) = x,$$
$$\hat{\varphi}_2(x, y) = y.$$

The gradients will be:

$$\frac{\partial \hat{\varphi}_0}{\partial x} = -1,$$
$$\frac{\partial \hat{\varphi}_0}{\partial y} = -1,$$
therefore the gradient is $\nabla \hat{\varphi}_0 = [-1, -1]$.
$$\frac{\partial \hat{\varphi}_1}{\partial x} = 1,$$
$$\frac{\partial \hat{\varphi}_1}{\partial y} = 0,$$
therefore the gradient is $\nabla \hat{\varphi}_1 = [1, 0]$.
$$\frac{\partial \hat{\varphi}_2}{\partial x} = 0,$$
$$\frac{\partial \hat{\varphi}_2}{\partial y} = 1,$$
therefore the gradient is $\nabla \hat{\varphi}_2 = [0, 1]$.

This corresponds to our snippet:

```
    gradientLocal(self, x):
        grad0 = numpy.array([-1, -1])
        grad1 = numpy.array([ 1, 0])
        grad2 = numpy.array([ 0, 1])
        return numpy.array([grad0, grad1, grad2])
```

Task 4. *Implement a function that assembles the system matrix and the forcing term. For this we will use the LinearLagrangeSpace. Use the attribute mapper from the LinearLagrangeSpace for index mapping. Remember that the gradients of the basis functions obtained from the LinearLagrangeSpace need to be converted to physical space using the jacobianInverseTransposed of the geometry.*

```
def assemble(space,force):
# storage for right hand side
rhs = numpy.zeros(len(space.mapper))
# storage for local matrix
localEntries = space.localDofs
localMatrix = numpy.zeros([localEntries,localEntries])
```

```
# data structure for global matrix using coordinate (COO) format
globalEntries = localEntries**2 * space.view.size(0)
value = numpy.zeros(globalEntries)
rowIndex, colIndex = numpy.zeros(globalEntries,int),
numpy.zeros(globalEntries,int)
# TODO: implement assembly of matrix and forcing term
...
# convert data structure to compressed row storage (csr)
matrix = scipy.sparse.coo_matrix((value, (rowIndex, colIndex)),
shape=(len(space.mapper),len(space.mapper))).tocsr()
return rhs,matrix
```

We use this snippet for the part we have to do:

```
    idx=0

    for e in space.view.elements:
        geo = e.geometry

        gE = space.mapper(e)
        localMatrix.fill(0.0)
        local_rhs = numpy.zeros(localEntries)

        for p in quadratureRule(e.type, order=2):

            w = p.weight * geo.integrationElement(p.position)

            phi = space.evaluateLocal(p.position)

            grad_phi_ref = space.gradientLocal(p.position)

            JinvT = geo.jacobianInverseTransposed(p.position)
            # transform the gradients from the reference element to the
               physical element.
            grad_phi_phys = numpy.array([JinvT @ grad_phi_ref[i] for i in range
               (localEntries)])


            for i in range(localEntries):
                for j in range(localEntries):
                    # mass matrix
                    localMatrix[i, j] += phi[i] * phi[j] * w


            # assemble the local right-hand side.
            x_phys = geo.toGlobal(p.position)
            f_val = force(x_phys)
            for i in range(localEntries):
                local_rhs[i] += f_val * phi[i] * w
        #assembling local matrix into global matrix
        for i in range(localEntries):
            global_i = gE[i]
```

```
            rhs[global_i] += local_rhs[i]
            for j in range(localEntries):
                global_j = gE[j]
                value[idx]   = localMatrix[i,j]
                rowIndex[idx] = global_i
                colIndex[idx] = global_j
                idx += 1
```

Consider the reference mapping:

$$F_E : \hat{E} \to E,$$

We use the four important tools:

1. Transformation formula for integrals. For $E \in \tau_h$:

$$\int_E y(x)dx = \int_{\hat{E}} y(F_E(\hat{x})))|detDF_E|dx.$$

2. Midpoint rule on the reference element:

$$\int_{\hat{E}} q(\hat{x})dx \approx q(\hat{S}_d)w_d$$

In our snippet, to approximate the integral over $\hat{E}$, we use a quadrature rule.

```
    w = p.weight * geo.integrationElement(p.position)
```

Here we multiply the weight by the integration factor $det(DF_E)$ evaluated at the quadrature point.

3. Basis functions via shape function transformation, we use:

```
    phi = space.evaluateLocal(p.position).
```

4. Computation of gradients, we transform the gradients from the reference element to the physical element.

```
    grad_phi_ref = space.gradientLocal(p.position)
            # inverse-transposed Jacobian at the quadrature point.
            JinvT = geo.jacobianInverseTransposed(p.position)
            # transform the gradients from the reference element to the
                physical element.
            grad_phi_phys = numpy.array([JinvT @ grad_phi_ref[i] for i in range
                (localEntries)])
```

In computing $M^E_{k,l}$ only the following elements are involved:

$$C(k,l) = \{(E,m,n) \in \tau_h \times \{0,...,d\} : \mathcal{M}_E(m) = k \wedge \mathcal{M}_E(n) = l\}$$

Then

$$\begin{aligned}
M_{kl} &:= \int_\Omega \varphi_k \varphi_l dx && \text{(by definition)} \\
&= \sum_{E\in\tau_h} \int_E \hat{\varphi}_k \hat{\varphi}_l dx && \text{(using mesh)} \\
&= \sum_{(E,m,n)\in C(k,l)} \int_{\hat{E}} \hat{\varphi}_k(\hat{x})\hat{\varphi}_l(\hat{x})|detDF_E|dx && \text{(localize)} \\
&= \sum_{(E,m,n)\in C(k,l)} \hat{\varphi}_k(\hat{S}_d)\hat{\varphi}_l(\hat{S}_d)|detDF_E(\hat{S}_d)|w_d && \text{(quadrature),}
\end{aligned}$$

with the following:

```
    for i in range(localEntries):
              for j in range(localEntries):

                  localMatrix[i, j] += phi[i] * phi[j] * w
```

Then we perform the assembly of right hand side, in computing b_l only the following elements are involved: $C(l) = \{(E, m) \in \tau_h \times \{0, ..., d\} : \mathcal{M}_E(m) = l\}$

$$
\begin{aligned}
b_l &= \int_\Omega u\varphi_l dx && \text{(by definition)} \\
&= \sum_{E\in\tau_h} \int_E u\varphi_l dx && \text{(use mesh)} \\
&= \sum_{(E,m)\in C(l)} \int_{\hat{E}} u(F_E(\hat{x}))\hat{\varphi_m}(\hat{x})|detDF_E|dx && \text{(localize)} \\
&= \sum_{(E,m)\in C(l)} u(F_E(\hat{S_d}))\hat{\varphi_m}(\hat{S_d})|detDF_E|w_d + err && \text{(quadrature)}
\end{aligned}
$$

In the code is:

```
    x_phys = geo.toGlobal(p.position)
           f_val = force(x_phys)
           for i in range(localEntries):
               local_rhs[i] += f_val * phi[i] * w
```

Finally, the mass matrix and local right hand side vector are assembled into the global system:

```
for i in range(localEntries):
    global_i = gE[i]
    rhs[global_i] += local_rhs[i]
    for j in range(localEntries):
        global_j = gE[j]
        value[idx] = localMatrix[i,j]
        rowIndex[idx] = global_i
        colIndex[idx] = global_j
        idx += 1
```

The global right hand side vector is updated in the first loop and in a second loop, entries are inserted into the `localMatrix`. These are stored in COO format and is later converted into CSR format.

Figure 0.1 shows the final plot that this code would give us after refining the grid.

We obtain the following output:
number of elements: 200 number of dofs: 121
number of elements: 800 number of dofs: 441
number of elements: 3200 number of dofs: 1681

Task 5. *Compute some error of the projection, i.e., in maximum difference between u_h and u at the center of each element, or some L^2 type error over the domain, e.g.,*

$$\sqrt{\int_\Omega |u - u_h|^2}.$$

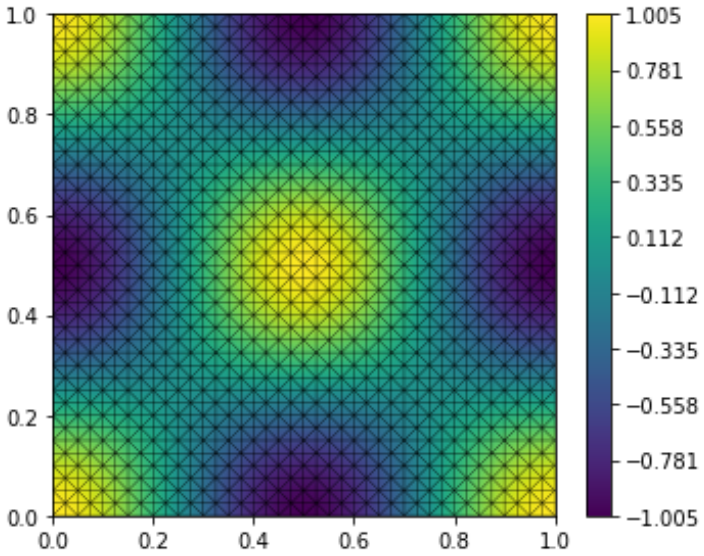

Figure 0.1: Task 4 Plot

What it the experimental order of convergence (EOC)? This is a simple procedure to test if a numerical scheme works. Assume, for example, that one can prove that the error e_h on a grid with grid spacing h satisfies

$$e_h \approx Ch^p$$

for some constant $C > 0$, then

$$log\frac{e_h}{e_H} \approx p\, log\frac{h}{H}$$

which can be used to get a good idea about the convergence rate p using the errors computed on two different levels with grid spacing h and H of a hierarchical grid.

We use the following snippet for the maximum error between u_h and u at the center of each element:

```
def computeMaxErrorAtCenters(view, space, dofs, exactFunction):
    max_error = 0.0
    for element in view.elements:
        geo = element.geometry
        center = geo.center
        u_exact = exactFunction(center)
        #  the reference center is at [1/3, 1/3]
        localCenter = numpy.array([1/3, 1/3])
        phi_center = space.evaluateLocal(localCenter)
        globalDofs = space.mapper(element)
        localValues = dofs[globalDofs]
        u_h_center = numpy.dot(localValues, phi_center)
        err = abs(u_exact - u_h_center)
        max_error = max(max_error, err)
    return max_error
```

While for the L^2 error we have this snippet:

```
  def computeL2Error(view, space, dofs, exactFunction):
```

```
    error_squared = 0.0
    for element in view.elements:
        geo = element.geometry
        globalDofs = space.mapper(element)
        for p in quadratureRule(element.type, order=2):
            w = p.weight * geo.integrationElement(p.position)
            x_phys = geo.toGlobal(p.position)
            u_exact = exactFunction(x_phys)
            phi = space.evaluateLocal(p.position)
            local_values = dofs[globalDofs]
            u_h = numpy.dot(local_values, phi)
            diff = u_exact - u_h
            error_squared += diff**2 * w
    return numpy.sqrt(error_squared)
```

Since our domain is:

```
    domain = cartesianDomain([0, 0], [1, 1], [10, 10])
```

The first mesh size h is 0.1, and using *view.hierarchicalGrid.globalRefine(2)* each time, h is halved.

We compute the error after we use *view.hierarchicalGrid.globalRefine(2)*, therefore we will denote $h_0 = 0.05$, $h_1 = 0.025$ and so on.

The L^2 error for the first two refinement levels is:

Refinement level 0	Refinement level 1
$e_0 = 4.645402e-03$	$e_1 = 1.142979e-03$

Computing:

$$\frac{e_0}{e_1} \approx 4.064,$$
$$\frac{h_0}{h_1} = \frac{0.05}{0.025} = 2,$$

Using logarithms:

$$log(\frac{e_0}{e_1}) \approx 1.4$$
$$log(\frac{h_0}{h_1}) \approx 0.6931$$

therefore

$$p \approx \frac{log(\frac{e_0}{e_1})}{log(\frac{h_0}{h_1})} \approx \frac{1.4}{0.6931} \approx 2.02.$$

As a consequence, the error between levels 0 and 1, it is reduced as $h^{2.02}$.

Performing the second refinement, we have the following.

Refinement level 1	Refinement level 2
$e_1 = 1.142979e-03$	$e_2 = 2.845499e-04$

Computing:

$$\frac{e_1}{e_2} \approx 4.02,$$
$$\frac{h_1}{h_2} = \frac{0.025}{0.0125} = 2,$$

Using logarithms:

$$log(\frac{e_1}{e_2}) \approx 1.39$$
$$log(\frac{h_1}{h_2}) \approx 0.6931$$

therefore

$$p \approx \frac{log(\frac{e_1}{e_2})}{log(\frac{h_1}{h_2})} \approx \frac{1.39}{0.6931} \approx 2.01.$$

The EOC is around 2 in each refinement. Therefore, the error decreases as h^2.
A similar computation is performed for the maximum error. We have the following values:

Refinement level 0	Refinement level 1	Refinement level 2
$e_0 = 1.069362e-02$	$e_1 = 2.725544e-03$	$e_2 = 6.844050e-04$

Therefore we will have the following EOC:

Refinement level 0 and 1	Refinement level 1 and 2
1.97	1.99

We conclude that the error decreases as h^2.

1 Appendix

task 4

```
import numpy
import scipy.sparse
import scipy.sparse.linalg
from scipy.sparse import csr_matrix
import numpy as np
from numpy import sin
from dune.fem.function import gridFunction
from numpy import sin, cos, pi
from dune.grid import cartesianDomain, yaspGrid
from dune.alugrid import aluConformGrid
from dune.geometry import quadratureRule

class LinearLagrangeSpace:
    def __init__(self,view):
        self.view=view
        self.mapper = view.mapper(lambda gt: 1 if gt.dim == 0 else 0)

        self.localDofs = 3
        self.points = numpy.array([
            [0, 0],
            [1, 0],
            [0, 1]
        ])
    def evaluateLocal(self, x):
        phi0 = 1-x[0]-x[1]
        phi1 = x[0]
        phi2 = x[1]
        return numpy.array([phi0,phi1,phi2])
        pass

    def gradientLocal(self, x):
        grad0 = numpy.array([-1, -1])
        grad1 = numpy.array([ 1, 0])
        grad2 = numpy.array([ 0, 1])
        return numpy.array([grad0, grad1, grad2])
        pass
#task 4
def assemble(space, force):
    # storage for right hand side
    rhs = numpy.zeros(len(space.mapper))
    # storage for local matrix
    localEntries = space.localDofs
    localMatrix = numpy.zeros([localEntries, localEntries])
    # data structure for global matrix using coordinate (COO) format
    globalEntries = localEntries**2 * space.view.size(0)
```

```
    value = numpy.zeros(globalEntries)
    rowIndex = numpy.zeros(globalEntries, int)
    colIndex = numpy.zeros(globalEntries, int)


    idx=0


    for e in space.view.elements:
        geo = e.geometry

        gE = space.mapper(e)

        localMatrix.fill(0.0) # initialize
        local_rhs = numpy.zeros(localEntries) #initialize


        for p in quadratureRule(e.type, order=2):

            w = p.weight * geo.integrationElement(p.position)

            phi = space.evaluateLocal(p.position)

            grad_phi_ref = space.gradientLocal(p.position)

            JinvT = geo.jacobianInverseTransposed(p.position)

            grad_phi_phys = numpy.array([JinvT @ grad_phi_ref[i] for i in range
                (localEntries)])


            for i in range(localEntries):
                for j in range(localEntries):

                    localMatrix[i, j] += phi[i] * phi[j] * w


            # Assemble the local right-hand side.
            x_phys = geo.toGlobal(p.position)
            f_val = force(x_phys)
            for i in range(localEntries):
                local_rhs[i] += f_val * phi[i] * w

        for i in range(localEntries):
            global_i = gE[i]
            rhs[global_i] += local_rhs[i]
            for j in range(localEntries):
                global_j = gE[j]
                value[idx]   = localMatrix[i,j]
                rowIndex[idx] = global_i
                colIndex[idx] = global_j
```

```
                idx += 1


    matrix = scipy.sparse.coo_matrix((value, (rowIndex, colIndex)),
                                     shape=(len(space.mapper), len(space.mapper)
                                        )).tocsr()
    return rhs, matrix

#main part
# First construct the grid
domain = cartesianDomain([0, 0], [1, 1], [10, 10])
view = aluConformGrid(domain)



# then the grid function to project
@gridFunction(view, order=3)
def u(p):
    x, y = p
    return numpy.cos(2*numpy.pi*x)*numpy.cos(2*numpy.pi*y)

u.plot(level=3)

# and then do the projection on a series of globally refined grids
for ref in range(3):
    space = LinearLagrangeSpace(view)
    print("number of elements:", view.size(0), "number of dofs:", len(space.
        mapper))
    rhs, matrix = assemble(space, u)
    dofs = scipy.sparse.linalg.spsolve(matrix, rhs)

    @gridFunction(view, order=1)
    def uh(e, x):
        indices = space.mapper(e)
        phiVals = space.evaluateLocal(x)
        localDofs = dofs[indices]
        return numpy.dot(localDofs, phiVals)

    uh.plot(level=1)
    view.hierarchicalGrid.globalRefine(2)
```

task 5

```
import numpy
import scipy.sparse
import scipy.sparse.linalg
from scipy.sparse import csr_matrix
from dune.fem.function import gridFunction
from dune.grid import cartesianDomain, yaspGrid
```

```
from dune.alugrid import aluConformGrid
from dune.geometry import quadratureRule

class LinearLagrangeSpace:
    def __init__(self, view):
        self.view = view

        self.mapper = view.mapper(lambda gt: 1 if gt.dim == 0 else 0)
        self.localDofs = 3
        self.points = numpy.array([
            [0, 0],
            [1, 0],
            [0, 1]
        ])

    def evaluateLocal(self, x):
        phi0 = 1 - x[0] - x[1]
        phi1 = x[0]
        phi2 = x[1]
        return numpy.array([phi0, phi1, phi2])
        pass
    def gradientLocal(self, x):
        grad0 = numpy.array([-1, -1])
        grad1 = numpy.array([ 1, 0])
        grad2 = numpy.array([ 0, 1])
        return numpy.array([grad0, grad1, grad2])
        pass
def assemble(space, force):
    # storage for right hand side
    rhs = numpy.zeros(len(space.mapper))
    # storage for local matrix
    localEntries = space.localDofs
    localMatrix = numpy.zeros([localEntries, localEntries])
    # data structure for global matrix using coordinate (COO) format
    globalEntries = localEntries**2 * space.view.size(0)


    value = numpy.zeros(globalEntries)
    rowIndex = numpy.zeros(globalEntries, int)
    colIndex = numpy.zeros(globalEntries, int)

    idx=0

    for e in space.view.elements:
        geo = e.geometry

        gE = space.mapper(e)

        localMatrix.fill(0.0) # initialize
        local_rhs = numpy.zeros(localEntries) #initialize
```

```
        for p in quadratureRule(e.type, order=2):

            w = p.weight * geo.integrationElement(p.position)

            phi = space.evaluateLocal(p.position)

            grad_phi_ref = space.gradientLocal(p.position)

            JinvT = geo.jacobianInverseTransposed(p.position)

            grad_phi_phys = numpy.array([JinvT @ grad_phi_ref[i] for i in range
                (localEntries)])


            for i in range(localEntries):
                for j in range(localEntries):

                    localMatrix[i, j] += phi[i] * phi[j] * w


            # Assemble the local right-hand side.
            x_phys = geo.toGlobal(p.position)
            f_val = force(x_phys)
            for i in range(localEntries):
                local_rhs[i] += f_val * phi[i] * w

        for i in range(localEntries):
            global_i = gE[i]
            rhs[global_i] += local_rhs[i]
            for j in range(localEntries):
                global_j = gE[j]
                value[idx]   = localMatrix[i,j]
                rowIndex[idx] = global_i
                colIndex[idx] = global_j

                idx += 1



    matrix = scipy.sparse.coo_matrix((value, (rowIndex, colIndex)),
                                     shape=(len(space.mapper), len(space.mapper)
                                        )).tocsr()
    return rhs, matrix


domain = cartesianDomain([0, 0], [1, 1], [10, 10])
view = aluConformGrid(domain)

@gridFunction(view, order=3)
def u(p):
    x, y = p
```

```
    return numpy.cos(2 * numpy.pi * x) * numpy.cos(2 * numpy.pi * y)

u.plot(level=3)


def computeL2Error(view, space, dofs, exactFunction):
    error_squared = 0.0
    for element in view.elements:
        geo = element.geometry
        globalDofs = space.mapper(element)
        for p in quadratureRule(element.type, order=2):
            w = p.weight * geo.integrationElement(p.position)
            x_phys = geo.toGlobal(p.position)
            u_exact = exactFunction(x_phys)
            phi = space.evaluateLocal(p.position)
            local_values = dofs[globalDofs]
            u_h = numpy.dot(local_values, phi)
            diff = u_exact - u_h
            error_squared += diff**2 * w
    return numpy.sqrt(error_squared)

def computeMaxErrorAtCenters(view, space, dofs, exactFunction):
    max_error = 0.0
    for element in view.elements:
        geo = element.geometry
        center = geo.center
        u_exact = exactFunction(center)
        #  the reference center is at [1/3, 1/3]
        localCenter = numpy.array([1/3, 1/3])
        phi_center = space.evaluateLocal(localCenter)
        globalDofs = space.mapper(element)
        localValues = dofs[globalDofs]
        u_h_center = numpy.dot(localValues, phi_center)
        err = abs(u_exact - u_h_center)
        max_error = max(max_error, err)
    return max_error


l2_errors = []
max_errors = []
mesh_sizes = []


#with the initial grid we had h=1/10, but if we start
#from view.hierarchicalGrid.globalRefine(2) it will be h=1/20

current_h = 1 / 20

num_refinements = 3 # number of refinement steps

for ref in range(num_refinements):
```

```
    view.hierarchicalGrid.globalRefine(2)
    print(f"\nRefinement step: {ref}")
    space = LinearLagrangeSpace(view)
    print("number of elements:", view.size(0), "number of dofs:", len(space.
        mapper))

    rhs, matrix = assemble(space, u)
    dofs = scipy.sparse.linalg.spsolve(matrix, rhs)


    @gridFunction(view, order=1)
    def uh(e, x):
        indices = space.mapper(e)
        phiVals = space.evaluateLocal(x)
        localDofs = dofs[indices]
        return numpy.dot(localDofs, phiVals)

    uh.plot(level=1)

    # Compute and store the errors.
    l2_err = computeL2Error(view, space, dofs, u)
    max_err = computeMaxErrorAtCenters(view, space, dofs, u)

    l2_errors.append(l2_err)
    max_errors.append(max_err)
    mesh_sizes.append(current_h)

    print(f"L2 error = {l2_err:.6e}, max center error = {max_err:.6e}, mesh
        size = {current_h:.3e}")



    current_h /= 2.0

print("\nEOC for L2 errors:")
for i in range(1, len(l2_errors)):
    e1, e2 = l2_errors[i-1], l2_errors[i]
    h1, h2 = mesh_sizes[i-1], mesh_sizes[i]

    frac1=e1/e2
    print(f" (e1/e2): {frac1:.2f}")
    loge1e2=numpy.log(e1/e2)
    logh1h2=numpy.log(h1/h2)
    rate = numpy.log(e1/e2) / numpy.log(h1/h2)
    print(f" log(e1/e2): {loge1e2:.2f}")
    print(f" log(h1/h2): {logh1h2:.2f}")
    print(f" Between levels {i-1} and {i}: EOC is {rate:.2f}")


print("\nEOC for max center errors:")
for i in range(1, len(max_errors)):
```

```
    e1, e2 = max_errors[i-1], max_errors[i]
    h1, h2 = mesh_sizes[i-1], mesh_sizes[i]
    loge1e2=numpy.log(e1/e2)
    logh1h2=numpy.log(h1/h2)
    rate = numpy.log(e1/e2) / numpy.log(h1/h2)
    frac1=e1/e2
    print(f" (e1/e2): {frac1:.2f}")
    print(f" log(e1/e2): {loge1e2:.2f}")
    print(f" log(h1/h2): {logh1h2:.2f}")
    print(f" Between levels {i-1} and {i}: EOC is {rate:.2f}")
```